Lesson 06 Demo 01

Creating a Theme Button Using the Context API

Objective: To implement a theme button using the Context API in React, providing a centralized theme management system for your application

Tools required: Node.js, Terminal, React app, and Visual Studio Code

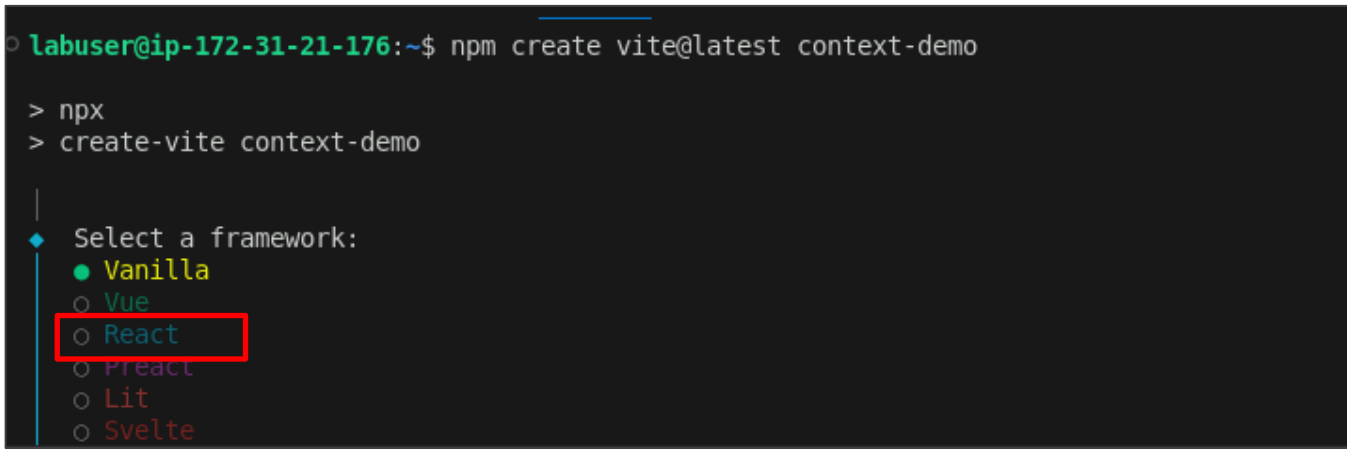
Prerequisites: Knowledge of creating a React app and understanding of the folder structure

Steps to be followed**:**

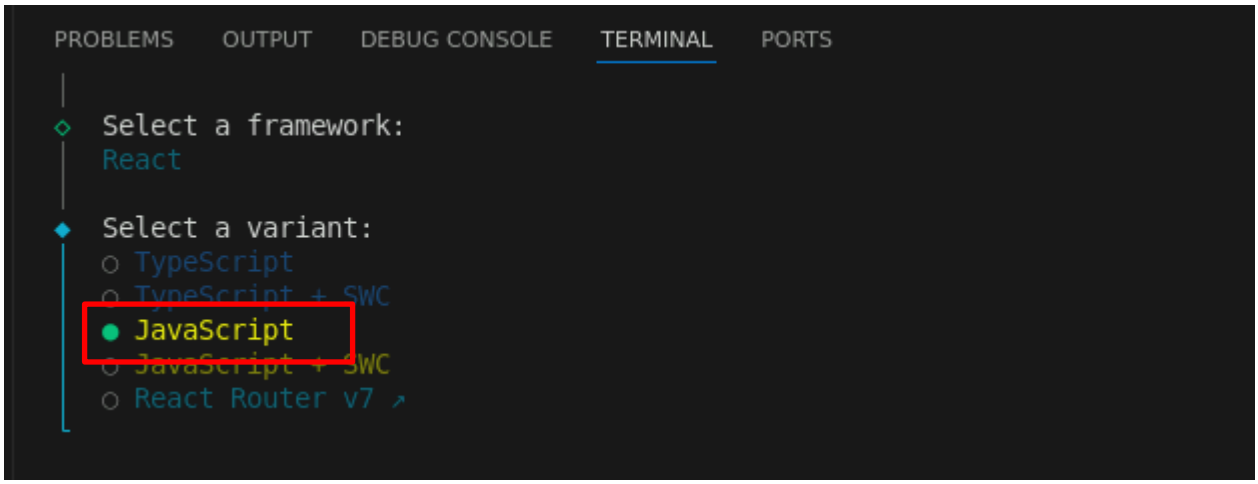
1. Create a new React app
2. Implement the Toolbar.js function
3. Wrap the Toolbar component in a ThemeProvider component
4. Update App.js to use the ThemeProvider component
5. Run the app

Step 1: Create a new React app

1.1 Open VS Code. In the terminal, run the following command to create a new React project using Vite and then select **React** as a framework:
npm create vite@latest context-demo



1.2 Select **JavaScript** as a variant

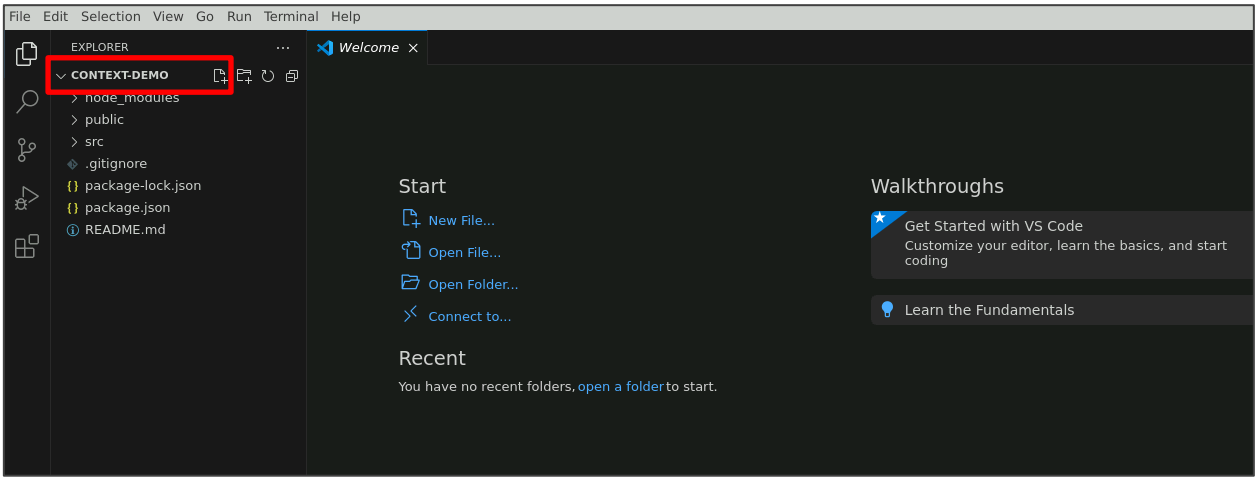


1.3 Run the **cd context-demo** command in the terminal

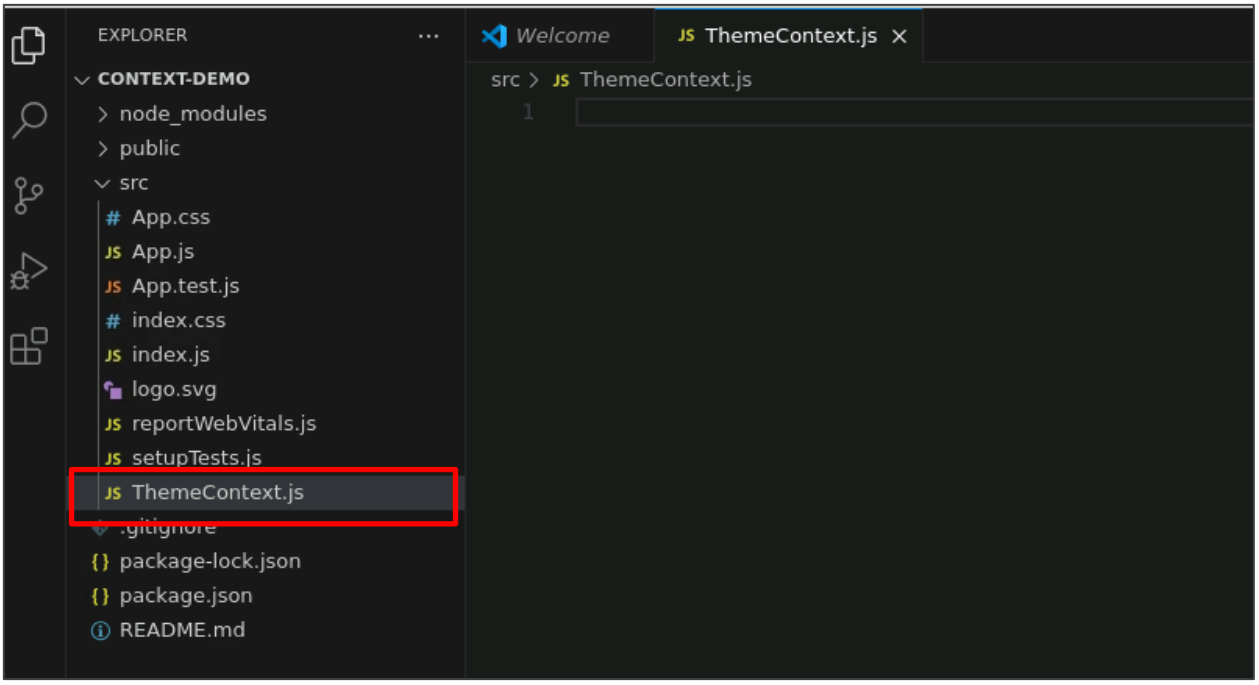
```
labuser@ip-172-31-45-231:~$ cd context-demo
labuser@ip-172-31-45-231:~/context-demo$
```

Note: This will change the current directory to the newly created **React** app directory.

1.4 Navigate to the newly created **CONTEXT-DEMO** directory in VS Code



1.5 Inside the src directory, create a JavaScript file named **ThemeContext.js**

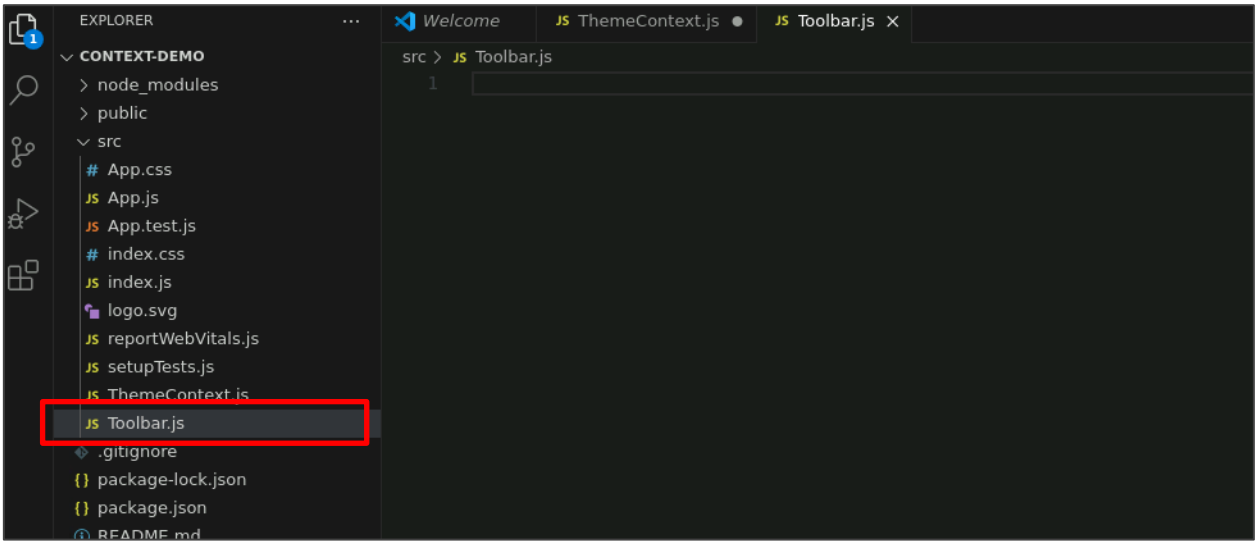


1.6 Inside the ThemeContext.js file, enter the code given below to create all required files inside a workspace:

import React, { createContext } from 'react';
const ThemeContext = createContext();
export default ThemeContext;

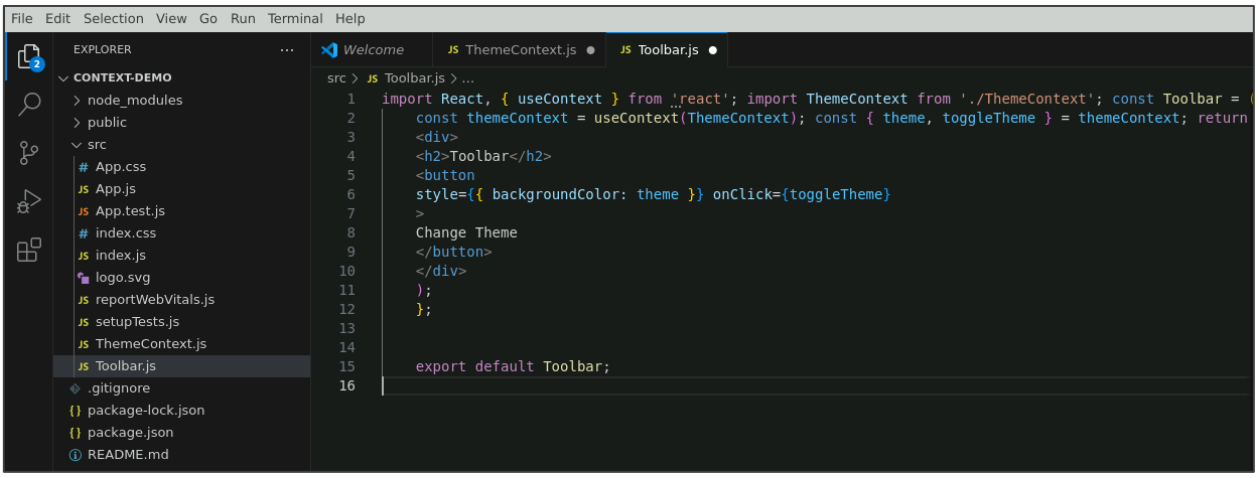
Step 2: Implement the Toolbar.js function

2.1 Create a file named **Toolbar.js** in the src directory



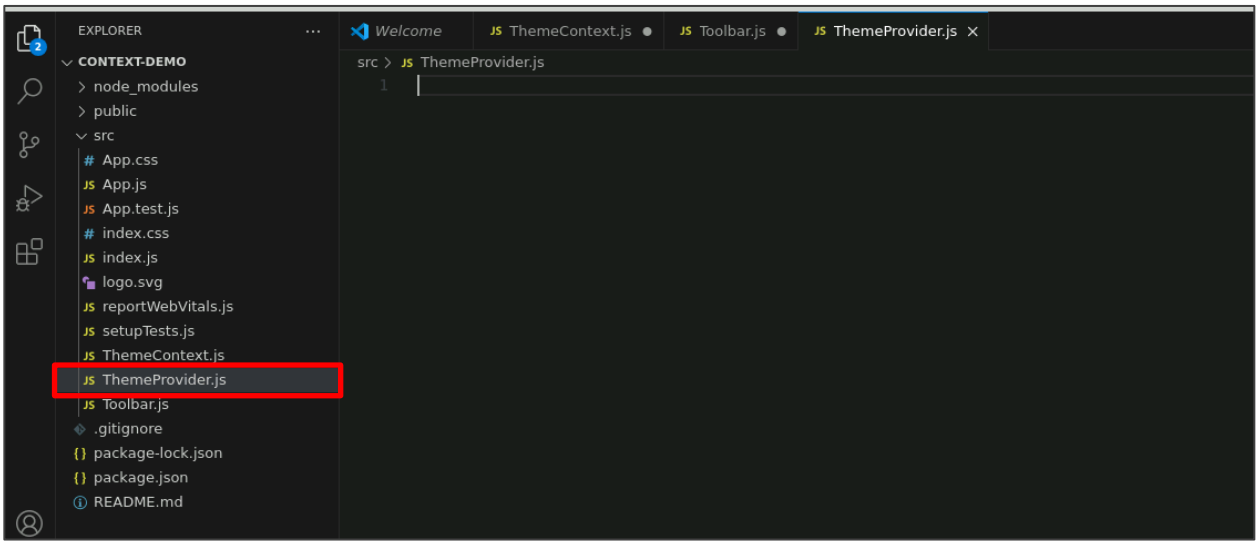
2.2 Enter the code below in the Toolbar.js file:

```
import React, { useContext } from 'react'; import ThemeContext from './ThemeContext'; const
    Toolbar = () => {
const themeContext = useContext(ThemeContext); const { theme, toggleTheme } =
    themeContext; return (
<div>
<h2>Toolbar</h2>
<button
style={{ backgroundColor: theme }} onClick={toggleTheme}
>
Change Theme
</button>
</div>
);
};
    export default Toolbar;
```



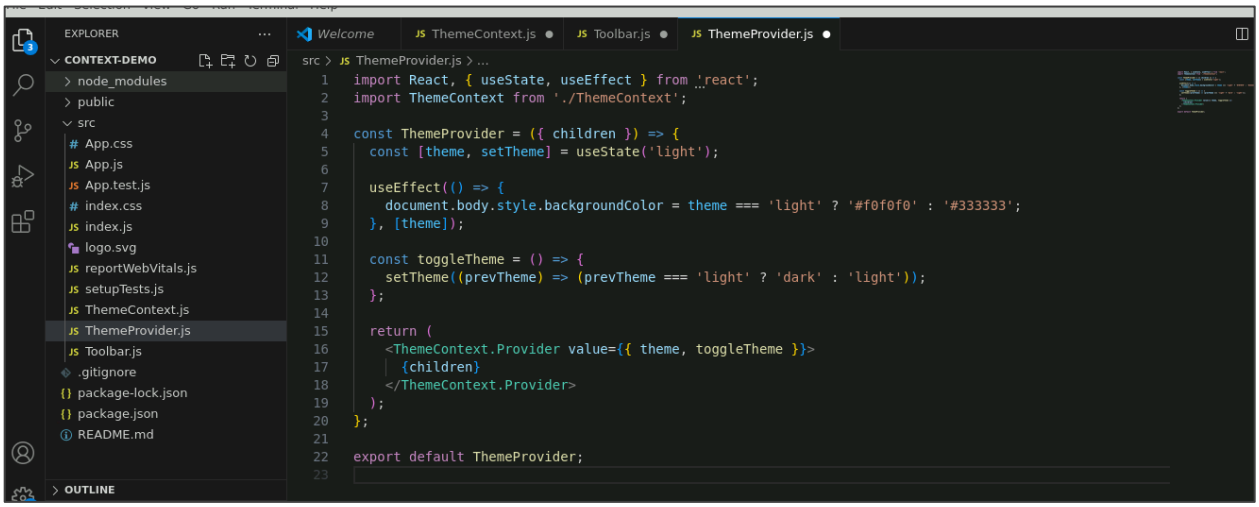
Step 3: Wrap the Toolbar component in a ThemeProvider component

3.1 In the src directory, create a new file named **ThemeProvider.js**



3.2 Enter the following code in the ThemeProvider.js file:

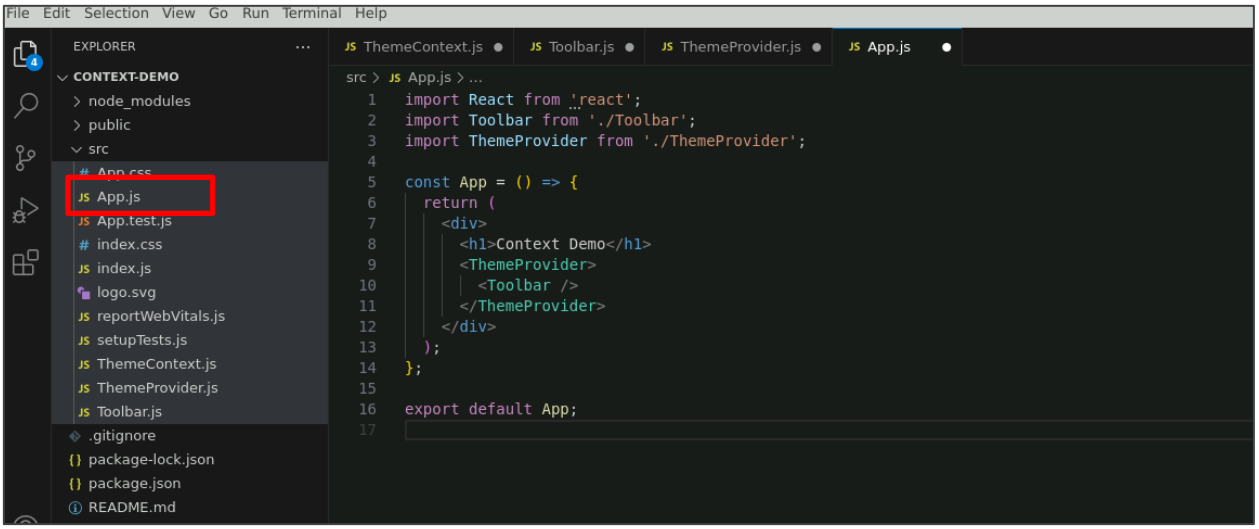
```
import React, { useState, useEffect } from 'react';
import ThemeContext from './ThemeContext';

const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');

  useEffect(() => {
    document.body.style.backgroundColor = theme === 'light' ? '#f0f0f0' : '#333333';
  }, [theme]);

  const toggleTheme = () => {
    setTheme((prevTheme) => (prevTheme === 'light' ? 'dark' : 'light'));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

export default ThemeProvider;
```



Step 4: Update App.js to use the ThemeProvider component

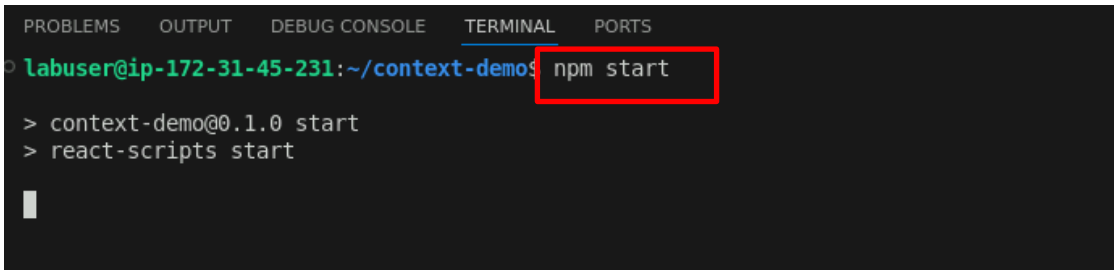
4.1 Open the App.js file and update it with the code below:

```
import React from 'react';
import Toolbar from './Toolbar';
import ThemeProvider from './ThemeProvider';

const App = () => {
  return (
    <div>
      <h1>Context Demo</h1>
      <ThemeProvider>
        <Toolbar />
      </ThemeProvider>
    </div>
  );
};

export default App;
```



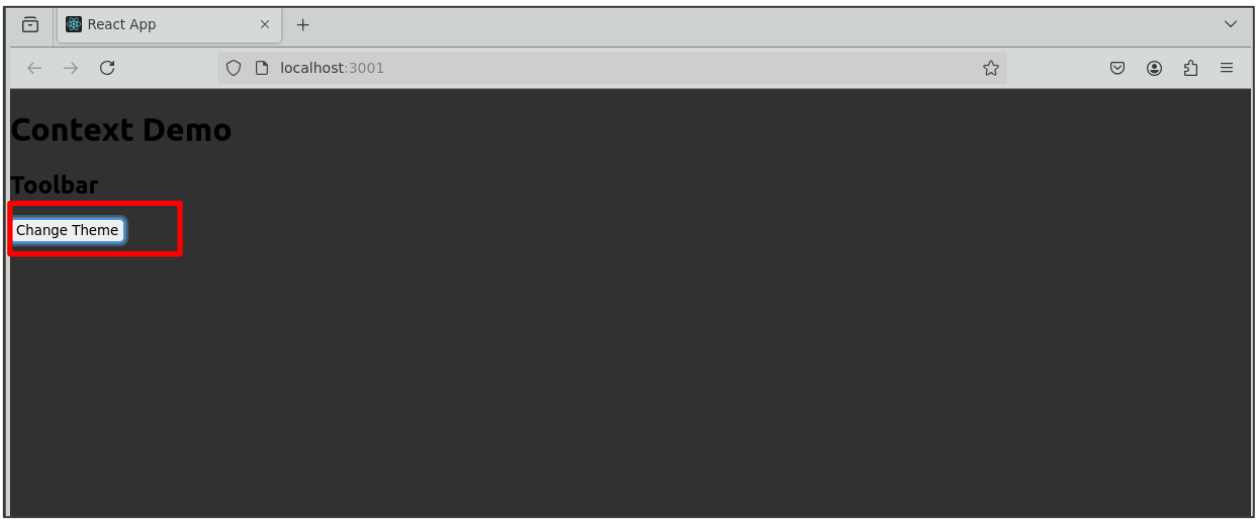
Step 5: Run the app

5.1 Navigate to the terminal and execute the command below to start the app:
npm start



Once the application starts, it will redirect you to the web page below:

5.2 Click on **Change Theme** to change the theme



By following these steps, you have successfully implemented a theme button using the Context API in React, providing a centralized theme management system for your application.